FICHE ORAL · DÉMO · 6 NOTES -> 6 PASSAGES

Expliquer la démo à Madame

Ouvrir, montrer chaque élément, expliquer le trajet des notes, puis répondre aux questions pièges.

Ouvrir la démo
cd "D:\ai-study-assistant\ai-study-assistant\modules\semantic_search"
.\.venv\Scripts\python.exe -m streamlit run demo_app.py
Puis ouvrir **http://localhost:8501** · rafraîchir avec **Ctrl+F5**

Ce que je montre à l'écran, élément par élément

Élément	Ce que je dis
Import (barre latérale)	Je peux remplacer les notes d'exemple par un JSON, TXT ou MD. Sans import, les 6 notes d'exemple restent.
Curseur « Nombre de résultats »	Je choisis le top-k : 5 par défaut. Il change le nombre de passages affichés.
Compteur « Notes »	Il affiche 6 : les 6 notes du fichier d'exemple sont chargées.
Compteur « Passages indexés »	Il affiche 6 : chaque note est assez courte pour tenir dans un seul passage.
Champ « Votre question »	Je pose une question en français, par exemple : comment éviter le sur-apprentissage ?
Bouton « Rechercher »	La question est encodée, comparée aux passages, puis les plus proches sont affichés.
Cartes de résultats	Je montre le titre, l'identifiant de la note, l'extrait et le score de similarité.
Bloc « Évaluer le moteur »	Je compare la recherche sémantique à une baseline mots-clés sur 6 requêtes annotées.
Tableau d'évaluation	Je lis Precision@k, Recall@k et MRR pour les deux moteurs.

Le trajet des 6 notes (ne pas confondre notes, passages et requêtes)

Étape	Ce qui se passe	Résultat
1. Charger	data/notes_exemple.json	6 notes = 6 textes
2. Découper	300 mots max, 50 de chevauchement	6 passages, car chaque note est courte
3. Encoder	MiniLM multilingue	6 vecteurs normalisés
4. Indexer	FAISS si possible, sinon NumPy	1 index vectoriel
5. Interroger	La question devient un vecteur	1 vecteur de question
6. Classer	Similarité cosinus + top-k	5 passages + scores

À retenir : 6 notes = 6 documents. 6 passages = 6 morceaux indexés. 6 requêtes = le jeu d'évaluation. 5 = le nombre de résultats affichés.

Les fichiers du projet à expliquer oralement

Fichier	Ce que je dis
demo_app.py	Interface Streamlit : import, curseur, compteurs, recherche, évaluation. Il appelle le moteur.
src/module.py	Cœur du projet : découpage, embeddings, index FAISS/NumPy, recherche et SearchResult.
src/corpus.py	Lecture et validation des notes JSON.
src/evaluate.py	Métriques Precision@k, Recall@k, MRR et baseline par mots-clés.
src/build_index.py	Commande CLI pour créer et sauvegarder l'index.
src/llm.py	Client Ollama optionnel. Il n'est pas utilisé dans la démo.
data/notes_exemple.json	Les 6 notes de cours utilisées par la démo.
data/evaluation_cases.json	Les 6 questions annotées pour l'évaluation.
tests/ + documentation	27 tests, README, prompts.md et rapport : preuves et traçabilité.

Script de 60 secondes

1. Ma démo est un moteur de recherche sémantique sur 6 notes de cours.
2. Sans import, les 6 notes d'exemple sont chargées automatiquement.
3. Chaque note est découpée en passages de 300 mots maximum, avec 50 mots de chevauchement.
4. Ici, 6 notes donnent 6 passages parce que chaque note fait moins de 300 mots.
5. Chaque passage devient un vecteur avec MiniLM ; la question devient aussi un vecteur.
6. Je classe les passages par similarité cosinus et j'affiche les 5 meilleurs avec leur score.
7. L'évaluation compare à une baseline mots-clés : MRR 0.889 contre 0.639, mais sur un petit corpus.
8. FAISS est prévu ; sa DLL est bloquée ici, donc NumPy prend le relais avec les mêmes résultats.

Phrases à mémoriser

- Le modèle ne cherche pas les mots : il compare le sens.
- 6 notes -> 6 passages : chaque note est plus courte que 300 mots.
- Le score est une similarité cosinus, pas une probabilité.
- FAISS accélère la recherche ; NumPy est le repli.
- Les 6 requêtes d'évaluation sont séparées des 6 notes.
- Claude.ai a aidé à écrire ; j'ai copié, exécuté et vérifié.
- Le sémantique n'est pas toujours meilleur : le corpus est minuscule.

Questions pièges et réponses courtes

Question	Réponse
Pourquoi 6 notes et 6 passages ?	Parce que chaque note fait moins de 300 mots. Une note plus longue produirait plusieurs passages.
Pourquoi pas un LLM pour chercher ?	Un embedding compare des vecteurs vite et de façon contrôlable. Un LLM génère du texte, il est plus lent et moins fiable pour classer.
À quoi sert FAISS ?	À accélérer la recherche des vecteurs proches. Ici il est bloqué par Windows, donc le code utilise NumPy.
Que mesurent Precision, Recall et MRR ?	Precision : part de résultats pertinents. Recall : part des bons passages retrouvés. MRR : position du premier bon résultat.
Le sémantique est-il meilleur ?	Pas toujours sur 6 notes. Ici le MRR est meilleur, mais le corpus est trop petit pour généraliser.
Où est l'IA dans le projet ?	Claude.ai a produit le code par prompts. sentence-transformers fait les embeddings. Ollama est optionnel.